1 Лекция 12. DP по древесному разложению: Independent Set и Dominating Set

1.1 Две задачи и один принцип динамики

Independent Set ищет множество попарно несмежных вершин. Во взвешенной версии у каждой вершины есть неотрицательный целый вес, и нужно максимизировать сумму весов. **Dominating Set** ищет минимальное множество D, такое, что каждая вершина либо лежит в D, либо имеет соседа в D. В доминирующем множестве соседние вершины разрешены. Эти задачи имеют разные ограничения, поэтому потребуют разных состояний динамики.

Например, на пути $a-b-c$ максимальное независимое множество при единичных весах — $\{a, c\}$ размера 2, а минимальное доминирующее — $\{b\}$ размера 1. Знание того, что вершина b не выбрана, достаточно для проверки независимости, но недостаточно для доминирования: надо также знать, есть ли уже выбранный сосед.

На входе вместе с графом дано корневое древесное разложение: все вершины покрыты мешками, каждое ребро имеет оба конца в одном мешке, мешки каждой вершины связны. Ширина t означает, что в мешке не больше $t + 1$ вершин. Используем nice-форму: пустые листья и корень; introduce добавляет вершину к мешку ребёнка, forget удаляет её, join соединяет двух детей с одинаковым мешком. Динамику вычисляем снизу вверх.

В каждом узле уже обработана часть графа под ним. Вершины текущего мешка — граница, через которую эта часть может взаимодействовать с оставшимся графом. Все подробности внутренних решений с одинаковым поведением на границе можно заменить одной лучшей стоимостью. Но сначала нужно точно определить это поведение; иначе формально похожие переходы могут оказаться неверными.

Вес записан двоично. Сложение и сравнение стоимостей учитываются с их битовой длиной; суммы не более чем n входных весов имеют полиномиальную длину. Ни один алгоритм здесь не перебирает все значения бюджета или веса.

1.2 Интерфейс и Independent Set

Пусть дано nice-разложение ширины t с q узлами. Обозначим V_a объединение мешков в поддереве узла a, включая B_a. Для каждого $S \subseteq B_a$ храним

$$F_a(S) = \max\{w(I) : I \subseteq V_a,\ I \text{ независимо},\ I \cap B_a = S\}.$$

Вес w неотрицателен и задан целыми числами двоично; невозможное состояние имеет значение $-\infty$. Весь вес частичного решения, включая мешок, уже учтён. Сепараторная лемма гарантирует, что взаимодействие с необработанной частью определяется ровно S.

Лист: $F_a(\varnothing) = 0$.

Introduce v с ребёнком b: если S не независимо, значение $-\infty$. Иначе при $v \notin S$ имеем $F_a(S) = F_b(S)$, а при $v \in S$:

$$F_a(S) = F_b(S \setminus \{v\}) + w(v).$$

Вершина v не имеет соседа в $V_b \setminus B_b$: мешок любого такого ребра нарушил бы связность мешков v или соседа. Поэтому проверка независимости внутри нового мешка достаточна.

Forget v: $F_a(S) = \max(F_b(S), F_b(S \cup \{v\}))$. Любое частичное решение либо содержит забытую вершину, либо нет; обе возможности разрешены и исчерпывают все решения.

Join с детьми b, c:

$$F_a(S) = F_b(S) + F_c(S) - w(S).$$

Пересечение $V_b \cap V_c$ равно B_a, а рёбер между исключительными частями нет. Два независимых решения с одинаковым S объединяются в независимое; любой родительский ответ ограничивается на эти два дочерних решения. Вес S посчитан дважды, поэтому вычитается один раз. Это доказывает равенство в обе стороны.

Ответ в пустом корне — $F_r(\varnothing)$. Сохраняя выбранный переход, восстанавливаем множество. Число состояний не больше $q2^{t+1}$, обработка состояния полиномиальна по t и размеру представления графа; достаточно оценки $2^t N^{O(1)}$. При явной матрице смежности получаем $O(n^2 + q2^{t+1}(t+1)^2)$ арифметических операций.

1.3 Таблица Independent Set на пути из трёх вершин

Пусть веса всех вершин пути $a - b - c$ равны 1. Используем цепочку мешков

$$\varnothing,\ \{a\},\ \{a,b\},\ \{b\},\ \{b,c\},\ \{c\},\ \varnothing.$$

После ввода a значения для $\varnothing$ и $\{a\}$ равны 0 и 1. В мешке $\{a,b\}$ значения $\varnothing, \{a\}, \{b\}$ равны 0,1,1, а $\{a,b\}$ невозможно из-за ребра.

После забывания a в мешке $\{b\}$ получаем $F(\varnothing) = 1$ (можно взять внутреннюю a) и $F(\{b\}) = 1$. После ввода c значение $F(\{c\}) = 2$: к решению $\{a\}$ добавлена c. Значение $F(\{b\}) = 1$, а $F(\{b,c\}) = -\infty$. После двух забываний пустой корень имеет значение 2 и восстанавливается ответ $\{a,c\}$.

Пустое пересечение с мешком не означает пустое решение: именно это показывает состояние $F_{\{b\}}(\varnothing) = 1$. Forget удаляет вершину только из интерфейса, а не из уже подсчитанного ответа.

1.4 Для доминирования одного бита мало

Dominating Set требует, чтобы каждая вершина была выбрана или соседствовала с выбранной. Нужно помнить и факт выбора, и выполненность требования.

Добавим introduce-edge узлы: мешок не меняется, оба конца вводимого ребра в нём, каждое ребро вводится ровно один раз и до забывания любого конца. Вставить такие узлы можно в место, где концы сосуществуют, перед исчезновением первого из них; ширина не увеличивается. Теперь G_a содержит вершины V_a и только рёбра, введённые в поддереве a.

Каждой вершине мешка назначаем точный статус: S — выбрана; D — не выбрана, но доминирована в G_a; U — не выбрана и ещё не доминирована. Все вершины $V_a \setminus B_a$ уже должны быть доминированы. Значение $A_a(\sigma)$ — минимум числа выбранных вершин во всём V_a при таком состоянии; невозможность — $+\infty$.

1.5 Переходы Dominating Set

В листе значение пустого состояния 0. В introduce-vertex узле новая вершина пока изолирована: её можно ввести со статусом S и добавить 1 к стоимости либо со статусом U без добавки. Статус D невозможен.

При introduce-edge uv перебираем дочерние состояния. Если u выбран, переводим v из U в D; симметрично для выбранного v. Остальные статусы не меняются. Несколько дочерних состояний могут дать одно родительское — берём минимум их стоимостей.

При forget разрешены только статусы S и D у исчезающей вершины. Статус U запрещён: все её инцидентные рёбра уже введены, и будущая часть не сможет её доминировать.

При join множество выбранных вершин мешка должно совпадать. Для невыбранной вершины статус в объединении — D, если хотя бы в одном ребёнке он D, иначе U. Стоимость равна сумме дочерних стоимостей минус число выбранных вершин мешка. Для одной вершины есть пять допустимых пар: $(S,S),(D,D),(D,U),(U,D),(U,U)$. Поэтому все join-переходы перечисляются за $5^{t+1}\,\mathrm{poly}(t)$ на узел; общая оценка $5^t N^{O(1)}$.

1.6 Минимальный пример отложенного доминирования

Рассмотрим единственное ребро ab. В пустом листе стоимость 0. Вводим a со статусом S: стоимость 1. Вводим b со статусом U: выбранных всё ещё 1, а ребро пока не обработано. При introduce-edge ab статус b меняется на D. Теперь a и b можно забыть, и пустой корень получит стоимость 1.

Если обе вершины вводились как U, ребро не доминирует ни одну из них: выбранных концов нет. Такое состояние не сможет пройти забывание. Если преждевременно разрешить статус D при вводе вершины или забыть U, алгоритм ошибочно допустит пустое доминирующее множество.

В корне $G_r = G$, мешок пуст, поэтому $A_r(\varnothing)$ — искомый минимум. При восстановлении сохраняем минимизирующие дочерние состояния; вводы со статусом S определяют выбранные вершины. В join одинаковая выбранная вершина двух ветвей включается в итоговое множество один раз.

1.7 Почему состояния достаточны

Внутренние вершины детей не имеют соседей в исключительной части другого ребёнка. Поэтому их требования нельзя исправить при join, и они уже выполнены по инварианту. Для мешка объединяется именно информация о доминировании через рёбра обеих частей. Любое глобальное решение ограничивается на допустимые дочерние состояния; любое согласованное объединение даёт допустимое родительское. Индукция по дереву завершает доказательство.

Это простая версия алгоритма, не оптимизация базы. В книге другая семантика третьего статуса даёт 4^t, а дальнейшие методы улучшают её. Нельзя переносить формулы между этими семантиками без нового доказательства.

Самопроверка: почему вес мешка вычитается, а забытая вершина не вычитается? Почему 3^{t+1} состояний не означает автоматически время 3^t?

1.8 Практика

Условия задач 12.1–12.5. Решения открывайте после своей попытки.

1.9 Источники и дополнительное чтение

Письменный источник: Cygan et al., *Parameterized Algorithms*, авторская копия 30.05.2016: §7.3.1, с. 162–167; §7.3.2, с. 168–171. PDF. Номера страниц печатные; номер страницы PDF больше на 16.

Изложение, поясняющие примеры и упражнения подготовлены для этого курса. Ссылки фиксируют происхождение результатов; для чтения конспекта видео не требуется. Необязательные видео собраны в программе.